

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_26

26. Null Object Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Null Object pattern.

Definition

It's not a GoF design pattern. Here's the definition from Wikipedia:

"In object-oriented computer programming, a null object is an object with no referenced value or with defined neutral ('null') behavior.

The null object design pattern describes the uses of such objects and their behavior (or lack thereof). It was first published in the Pattern Languages of Program Design book series."

Concept

The pattern can implement a "do-nothing" relationship, or it can provide a default behavior when an application encounters a null object instead of a real object. Using this pattern, our core aim is to make a better solution by avoiding a "null objects check" or a "null collaborations check" through `if` blocks. Normally, you encapsulate the absence of an object by providing a default behavior that does nothing. The basic structure of this pattern is shown in Figure 26-1.

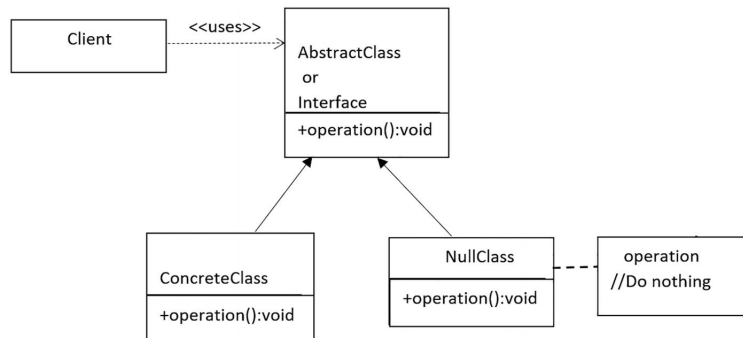


Figure 26-1 The basic structure of a Null Object pattern

In this chapter, I begin with a program that seems to be ok, but it has a serious potential bug. When you analyze the bug with a potential solution, the need for the Null Object Pattern will become clear. So, let's jump to the next section.

A Faulty Program

Let's assume that you have two different types of vehicles, `Bus` and `Train`, and a client can pass in different inputs. These clients can type `b` to create a `Bus` object, `t` to create a `Train` object, or `exit` to quit the application. The following program demonstrates this. This program runs smoothly when the inputs are valid, but the potential bug will be revealed when they supply an invalid input. Here's the faulty program:

```

package jdp3e.null_object;
import java.util.Scanner;
interface Vehicle {
    void travel();
}
class Bus implements Vehicle {
    public static int busCount = 0;
    public Bus() {
        busCount++;
    }

    @Override
    public void travel() {
        System.out.println("Let us travel on a bus.");
    }
}
class Train implements Vehicle {
    public static int trainCount = 0;
    public Train() {
        trainCount++;
    }
    @Override
    public void travel() {
        System.out.println("Let us travel on a train.");
    }
}
class NeedForNullObjectPattern {
    public static void main(String[] args) {
        System.out.println("***This program demonstrates
            the need for a Null Object pattern.***\n");
        String input = null;
        Vehicle vehicle = null;
        Boolean flag = true;
        int totalObjects = 0;
        Scanner scanner = new Scanner(System.in);
        try {

            while (flag) {
                System.out.println("Enter your choice(
                    Type 'b' for Bus, 't' for Train ) ");
                input = scanner.next();
                switch (input.toLowerCase()) {
                    case "b":
                        vehicle = new Bus();
                        vehicle.travel();
                        break;
                    case "t":
                        vehicle = new Train();
                        vehicle.travel();
                        break;
                    case "exit":
                        flag = false
                }
            }

            System.out.println("Closing the application.");
            break;
            default:
                System.out.println("Invalid input.");
                System.out.println("Using the vehicle that was created earlier.
                    vehicle.travel();
                }
            totalObjects = Bus.busCount +

```

```

        Train.trainCount;
        System.out.println("Number of objects
                           created:" + totalObjects);
    }
    } finally {
        scanner.close();
    }
}
}

```

Output

Let's focus on the potential bug. Here is the output with some valid and invalid inputs:

```

***This program demonstrates the need for a Null Object pattern.***
Enter your choice( Type 'b' for Bus, 't' for Train )
b
Let us travel on a bus.
Number of objects created:1
Enter your choice( Type 'b' for Bus, 't' for Train )
e
Invalid input

.
Using the vehicle that was created earlier.
Let us travel on a bus.
Number of objects created:1
Enter your choice( Type 'b' for Bus, 't' for Train )
t
Let us travel on a train.
Number of objects created:2
Enter your choice( Type 'b' for Bus, 't' for Train )
e
Invalid input.
Using the vehicle that was created earlier.
Let us travel on a train.
Number of objects created:2
Enter your choice( Type 'b' for Bus, 't' for Train )
b
Let us travel on a bus.
Number of objects created:3
Enter your choice( Type 'b' for Bus, 't' for Train )
exit
Closing the application.
Number of objects created:3

```

An Unwanted Input

Let's assume that by mistake, a user supplied an invalid input, say e as the first input, as shown here:

```

***This program demonstrates the need for a Null Object pattern.***
Enter your choice( Type 'b' for Bus, 't' for Train )
e

```

You receive the following runtime exception:

```

Invalid input.
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "jdp3e.null_ob:
    at jdp3e.null_object.NeedForNullObjectPattern.main(NeedForNullObjectPattern.java:6

```

```
Using the vehicle that was created earlier.
```

The Potential Fix

The immediate remedy that may come to mind is to do a null check before invoking the operation, as shown here:

```
if (vehicle != null){  
    vehicle.travel();  
}
```

Analysis

The prior solution will work in this case. But think of an enterprise application. When you do null checks like this and put `if` conditions like this in each possible place, you make your code dirty. This kind of code is difficult to maintain. The concept of the Null Object pattern is useful in similar cases.

Point to Note

A particular language can have special support in a similar context. For example, C# programmers can avoid the null check using the null conditional operator like the following: `vehicle?.Travel();`

This operator is available in C# 6 and later versions only. Still, it can be beneficial to look into the implementation details of the Null Object pattern. For example, when you use the Null Object pattern, instead of doing nothing, you can supply a default behavior (that suits your application best) for those null objects.

Another point: in a similar program in C#, following the naming convention, the method name should start with a capital letter, so I use the method `Travel()` instead of `travel()`.

Real-Life Example

A washing machine can wash properly if there is a smooth water supply without any internal leakage. But suppose, on one occasion, you forget to supply the water before you start washing the clothes, but you pressed the button that initiates washing the clothes. The washing machine should not damage itself in such a situation so it may beep an alarm to get your attention and indicate that there is no water supply at the moment.

Computer World Example

Assume that in a client-server architecture, the server does a calculation based on client input. The server needs to be intelligent enough so that it will not initiate any calculation unnecessarily. Prior to processing the input, it may want to do a cross-verification to ensure whether it needs to start the calculation at all or just ignore invalid input. You may notice the use of the Command pattern with the Null Object pattern in such a case.

In Java, you may have seen the use of various adapter classes in the `java.awt.event` package. These classes can be considered as closer examples to the Null Object pattern. For example, consider the

`MouseMotionAdapter` class. It is an abstract class but it contains methods with empty bodies like `mouseDragged(MouseEvent e) {}` and `mouseMoved(MouseEvent e) {}`. However, since the adapter class is tagged with the `abstract` keyword, you cannot directly create objects of the class.

Basically, in an enterprise application, you can avoid a big number of **null checks and if/else blocks** using this design pattern. The following implementation will give you a quick overview of this pattern.

Implementation

Let's modify the faulty program we discussed before. You handle the invalid input through a `NullVehicle` object this time. So, if by mistake the user supplies any invalid data (in other words, any input other than *a*, *b*, or *exit* in this case), the application will do nothing. It will ignore those invalid inputs through a `NullVehicle` object, which actually does nothing. The class is defined as follows:

```
class NullVehicle implements Vehicle {
    // Early initialization
    private static NullVehicle instance = new NullVehicle();
    public static int nullVehicleCount;
    // The constructor is private to prevent the use of "new"
    private NullVehicle() {
        nullVehicleCount++;
        System.out.print("A null vehicle object created.");
    }
    // Global point of access.
    public static NullVehicle getInstance(){
        return instance;
    }
    @Override
    public void travel(){
        // Do nothing
    }
}
```

You apply the concept of the Singleton design pattern when you create a `NullVehicle` object. Because there can be an infinite number of invalid inputs, in the following example you do not want to create the `NullVehicle` object repeatedly. Once you have a `NullVehicle` object, you want to reuse that object.

Points to Remember

For a null object method, you need to return whatever seems sensible as a default. In this example, you cannot travel in a vehicle that does not exist. So, it makes sense that for `NullVehicle` class, the `travel()` method does nothing.

Class Diagram

Figure [26-2](#) shows the class diagram.

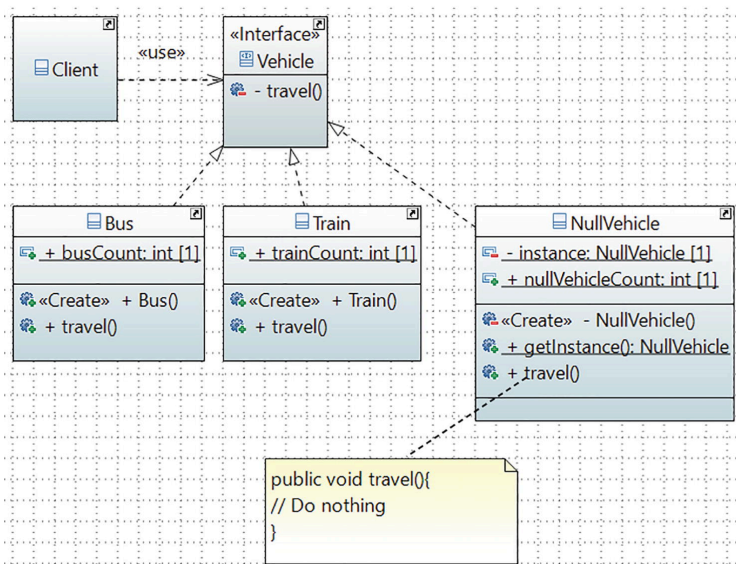


Figure 26-2 Class diagram

Package Explorer View

Figure 26-3 shows the high-level structure of the program.

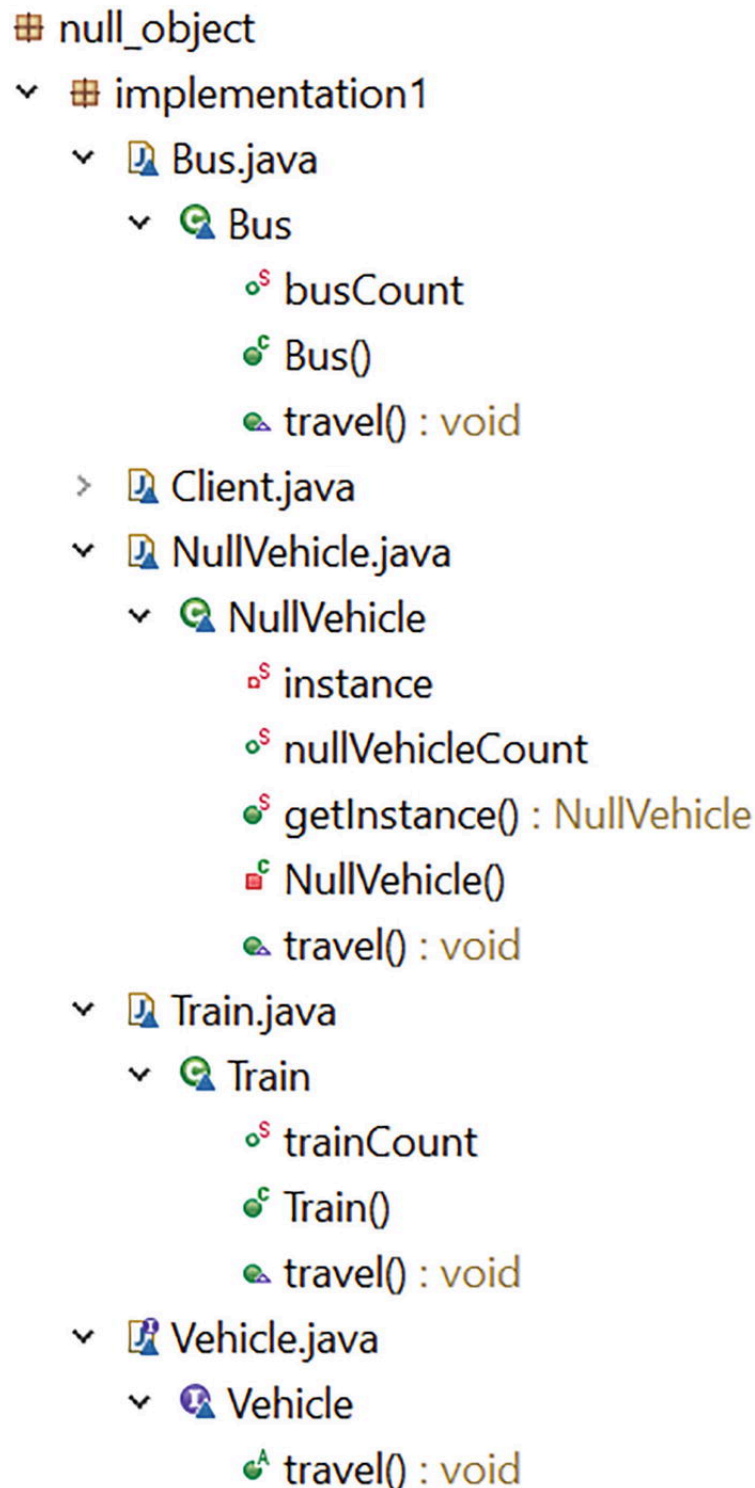


Figure 26-3 Package Explorer view

Demonstration

All parts of the program are stored inside the package

`jdp3e.null_object.implementation1`. Here's the complete implementation:

```
// Vehicle.java
interface Vehicle {
    void travel();
}

// Bus.java
class Bus implements Vehicle {
    public static int busCount = 0;
    public Bus() {
        busCount++;
    }
}
```

```

        @Override
        public void travel() {
            System.out.println("Let us travel on a bus.");
        }
    }
}

// Train.java
class Train implements Vehicle {
    public static int trainCount = 0;
    public Train() {
        trainCount++;
    }
    @Override
    public void travel() {
        System.out.println("Let us travel on a train.");
    }
}

// NullVehicle.java
class NullVehicle implements Vehicle {
    // Early initialization
    private static NullVehicle instance = new NullVehicle();
    public static int nullVehicleCount;
    // The constructor is private to prevent the use of "new"
    private NullVehicle() {
        nullVehicleCount++;
        System.out.print("A null vehicle object is created.");
    }
    // Global point of access.
    public static NullVehicle getInstance(){
        return instance;
    }
    @Override
    public void travel(){
        // Do nothing
    }
}

// Client.java
import java.util.Scanner;
class Client {
    public static void main(String[] args) {
        System.out.println("***The Null Object pattern demonstration.***\n");
        String input = null;
        Vehicle vehicle = null;
        Boolean flag = true;
        int totalObjects = 0;
        Scanner scanner = new Scanner(System.in);
        try {
            while (flag) {
                System.out.println("Enter your choice(
                    Type 'b' for Bus, 't' for Train ) ");
                input = scanner.next();
                switch (input.toLowerCase()) {
                    case "b":
                        vehicle = new Bus();
                        break;
                    case "t":
                        vehicle = new Train();
                        break
                }
                totalObjects++;
            }
        } catch (Exception e) {
            System.out.println("Invalid input. Please try again.");
        }
        System.out.println("Total objects created: " + totalObjects);
    }
}

```



```

        flag = false;
        System.out.println("Closing the application.");
        break;
    default:
        System.out.println("Invalid input.");
        vehicle=NullVehicle.getInstance();
    }
    // There is no need to do a null check now.
    vehicle.travel();
    totalObjects =
        Bus.busCount +
        Train.trainCount +
        NullVehicle.nullVehicleCount;
    System.out.println("Number of objects
        created:" + totalObjects);
    }
    } finally {
        scanner.close();
    }
}
}

```

Output

Here's the output:

```

***The Null Object pattern demonstration.***
Enter your choice( Type 'b' for Bus, 't' for Train )
a
Invalid input.
A null vehicle object is created.
Number of objects created:1
Enter your choice( Type 'b' for Bus, 't' for Train )
b
Let us travel on a bus.
Number of objects created:2
Enter your choice( Type 'b' for Bus, 't' for Train )
t
Let us travel on a train.
Number of objects created:3
Enter your choice( Type 'b' for Bus, 't' for Train )
xyz
Invalid input.
Number of objects created:3
Enter your choice( Type 'b' for Bus, 't' for Train )
exit
Closing the application.
Number of objects created:3

```

Analysis

I draw your attention to the following points:

- Invalid inputs and their effects are shown in bold.
- There is only one null vehicle object. This count does not increase due to invalid input.
- You did not perform a null check. Still, the program execution is not interrupted because of invalid user input.
- Following the DRY principle, there is a single occurrence of `vehicle.travel()`; but in the faulty program (which was shown earlier), this line appears multiple times inside the `switch` block.

Q&A Session

26.1 At the beginning of the implementation, I see an additional object is created. Is this intentional?

To save computer memory/storage, I follow a Singleton design pattern that supports early initialization when I construct the `NullVehicle` class. This is why you see this additional object. You do not want to create a `NullVehicle` object for each invalid input because your application may receive a large number of invalid inputs. If you do not guard against this situation, a huge number of `NullVehicle` objects may reside in the system (which is basically useless), and they can occupy a big amount of computer memory, which in turn can cause some unwanted side effects. (For example, the system may become slow, the application's response time may increase, etc.)

26.2 When should you use this pattern?

This pattern can be useful in the following cases:

- You do not want to encounter a `NullPointerException` (for example, if by mistake you try to invoke a method from a null object).
- You like to ignore lots of null checks in your code.
- You want to make your code cleaner and easily maintainable.
- Finally, I want to draw your attention to another interesting point. The Null Object pattern can be useful in another context. Consider the following segment of code:

```
System.out.println("Doing an additional test");
System.out.println("We'll iterate over the list of vehicles.");
List<Vehicle> vehicleList = new ArrayList<Vehicle>();
vehicleList.add(new Bus());
vehicleList.add(new Train());
vehicleList.add(null);
```

When you use this code segment, you'll receive `java.lang.NullPointerException` again. But if you replace

```
vehicleList.add(null);
```

with

```
vehicleList.add(NullVehicle.getInstance());
```

there is no runtime exception so you can loop through easily. This is another important usage of this pattern.

26.3 What are the challenges associated with the Null Object pattern?

You need to be aware of the following scenarios:

- Most often, you may want to find and fix the root cause of a failure. So, if you throw a `NullPointerException`, that can work better for you. You can always handle those exceptions in a `try-catch` block or in a `try-catch-finally` block and update the log information accordingly.
- The Null Object pattern basically helps you implement a default behavior when you unconsciously want to deal with an object that is not

present at all. But trying to supply such a default behavior may not always be appropriate.

- Incorrect implementations of the Null Object pattern can suppress the true bug that may appear as normal in your program execution.

26.4 Looks like null objects are working like proxies. Is this understanding correct?

No. In general, proxies act on real objects at some point in time and they can be used to provide a special behavior. But a null object should not do any such thing.

26.5 The Null Object Pattern is always associated with `NullPointerException`. Is this understanding correct?

The concept is the same, but the exception name can be different or language specific. For example, in Java, you use this pattern to guard against `java.lang.NullPointerException` but in C#, you use it to guard against `System.NullReferenceException`.

Previous chapter

< [Part III. Additional Design Patterns](#)

Next chapter

[27. MVC Pattern](#) >